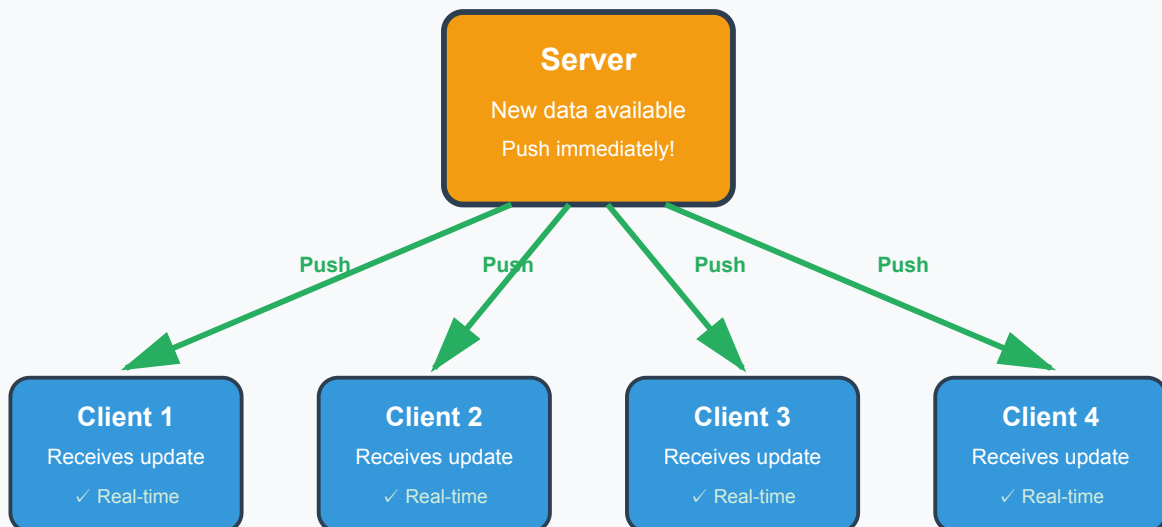


Push vs Pull in System Design

 systemdesignschool.io/fundamentals/push-vs-pull

Push Model: Server Initiates Data Transfer



Push vs Pull

We explored three dataflow patterns: databases, service calls, and message queues. But within these patterns, data moves in two ways. Either the producer pushes data to consumers, or consumers pull data from the producer. This choice affects latency, scalability, and resource usage.

Push Model

The server initiates data transfer. When new data appears, the server sends it to clients immediately without waiting for a request. A chat application demonstrates this. When Alice sends a message, the server pushes it to Bob's device. Bob does not poll for new messages. The server maintains connections to all online clients and pushes updates as they occur.

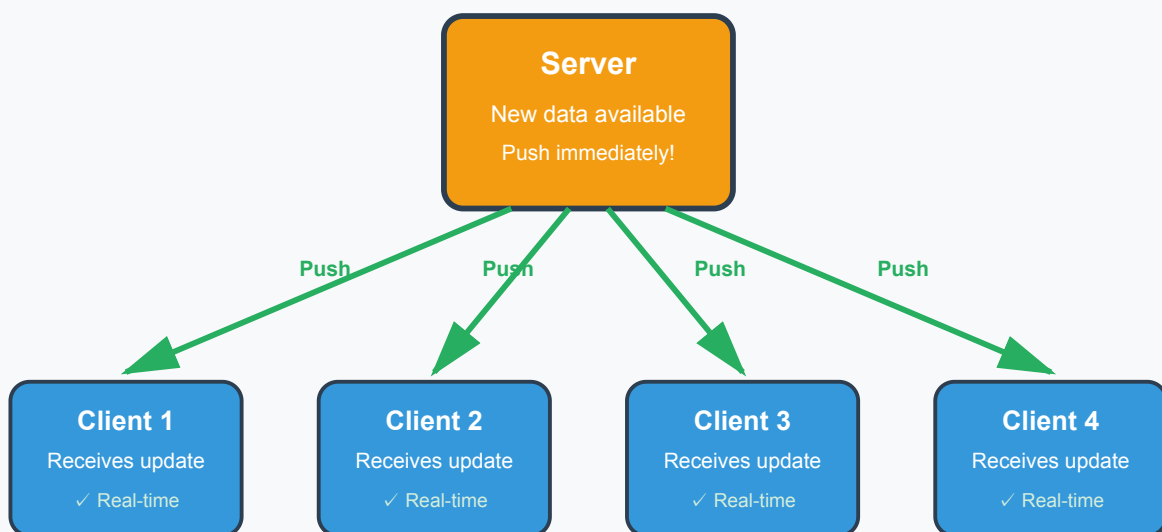
WebSockets enable push in web applications. The client opens a persistent connection. The server keeps it alive and sends data whenever events happen. Stock trading platforms push price updates. News apps push breaking stories. IoT sensors push measurements to central systems.

Push delivers data with minimal latency. Updates arrive as soon as they are produced. Users see changes in real time. No polling delay exists between when data becomes available and when clients receive it.

But push consumes server resources. The server tracks all connected clients. It maintains open connections and manages state for each one. A million concurrent users mean a million connections to maintain. Memory usage and network overhead scale with client count.

Push works best when updates are frequent and unpredictable. Live sports scores change constantly. Financial data updates every second. Chat messages arrive randomly. If clients need immediate notification, push avoids polling waste.

Push Model: Server Initiates Data Transfer



Pull Model

The client initiates data requests. Clients ask for updates at intervals or on demand. The client polls the server: "Any new data?" The server responds with whatever is available. Web browsers work this way. Click a link. The browser requests a page. The server responds.

API calls use pull. A mobile app calls `GET /notifications` every 30 seconds. The server returns new notifications if any exist. If nothing changed, it returns an empty list. The client controls request frequency.

Pull gives clients control over timing and rate. A client processing data slowly can reduce polling frequency. A client catching up can increase it. No server-side tracking of client state is needed. The server simply responds to incoming requests.

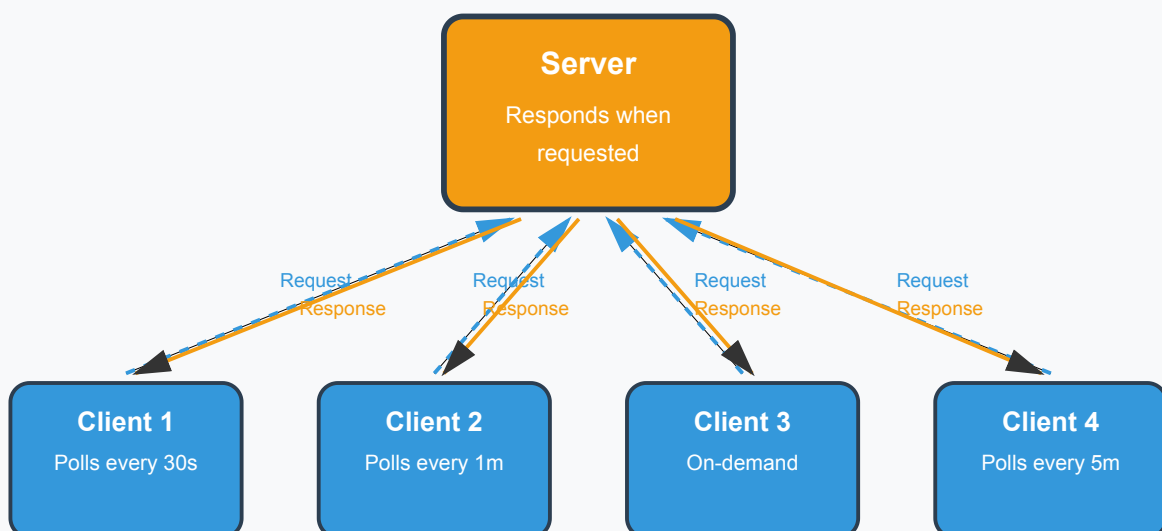
Pull adds latency. New data sits on the server until the next poll. If a client polls every minute, updates lag by up to a minute. Reducing the polling interval increases freshness but wastes resources. Polling every second when updates come every hour generates 3,600 unnecessary requests.

Empty polls waste bandwidth and compute. The client asks for new data. The server checks. Nothing new exists. Send an empty response. Both sides burned resources for no value. Under heavy polling, most requests return no data.

Pull scales better on the producer side. The server does not track clients. It just answers requests as they arrive. No persistent connections to maintain. A stateless server handles millions of sporadic requests more easily than millions of open connections.

Use pull when updates are infrequent or when clients have varying consumption rates. Configuration data changes rarely. An app can poll for config updates every hour. Background sync jobs run on their own schedule. Pull lets them fetch data when ready.

Pull Model: Client Initiates Data Requests



Comparing Push and Pull

Aspect	Push	Pull
Latency	Immediate	Delayed by polling interval
Server Resources	High (maintain connections)	Low (stateless responses)
Client Control	Limited	Full control over timing
Network Efficiency	High (no empty polls)	Low (many empty polls)
Scalability	Challenging with many clients	Easier to scale horizontally
Best For	Real-time updates	Infrequent or scheduled updates

Real systems blend both models. Email uses pull. Clients check for new messages periodically. But mobile push notifications alert users immediately. The combination gives real-time awareness without constant polling.

Long polling bridges push and pull. The client sends a request. The server holds it open until new data arrives. When data becomes available or a timeout expires, the server responds. The client immediately sends another request. This provides near-real-time updates without WebSocket complexity.

Server-sent events (SSE) are another hybrid. The client opens a connection with a standard HTTP request. The server keeps it open and streams updates. Unlike WebSockets, SSE is unidirectional. The server pushes. The client cannot send messages back over the same connection.

Choose based on update frequency, latency requirements, and scale. High-frequency real-time data favors push. Sporadic or scheduled data favors pull. That leads into how Twitter combines both models to deliver timeline updates efficiently at massive scale.